République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
TEK-UP University

Projet de Cours

Sécurité Informatique

Deuxième Année du Cycle d'Ingénieur Informatique

Sécurité des Systèmes Informatiques et des Réseaux

ING-4-J-SSIR

Antivirus

Réalisé par :
Oussama Hamada
Hamza Kalai
Rayen Hassen

Encadré par :
FAKER NAOUFE

Type de projet :	Application de Sécurité
Fonctionnalités :	Scan de fichiers, Réputation IP/Domaine, Analyse comportementale
Technologies :	Python 3, PyQt5, API VirusTotal
Architecture :	Multithread

Année Universitaire : 2024 - 2025

Date de validation : 6 Janvier 2026

Table des matières

1 Cahier des Charges (Version Finale) 3
1.1 Informations du Projet . . . 3
1.2 Introduction Générale et Cadre du Mini Projet . . . 3
1.3 Objectifs . . . 3
1.4 Besoins Fonctionnels . . . 4
1.4.1 BF-01 : Scan de fichiers et dossiers . . . 4
1.4.2 BF-02 : Calcul de signatures . . . 4
1.4.3 BF-03 : Analyse de réputation IP/Domaine . . . 4
1.4.4 BF-04 : Intégration API VirusTotal . . . 4
1.4.5 BF-05 : Surveillance comportementale . . . 4
1.4.6 BF-06 : Journalisation . . . 4
1.4.7 BF-07 : Interface graphique . . . 4
1.5 Besoins Non Fonctionnels . . . 5
1.5.1 Sécurité . . . 5
1.5.2 Performance . . . 5
1.5.3 Fiabilité . . . 5
1.5.4 Simplicité . . . 5
1.6 Environnements de Développement . . . 5
1.6.1 Environnement Matériel . . . 5
1.6.2 Environnement Logiciel . . . 5
1.6.3 Bibliothèques Python Utilisées . . . 5

2 Cas d'Utilisation (Use Cases) 7
2.1 Diagramme des Cas d'Utilisation . . . 7
2.2 Description des Cas d'Utilisation . . . 7
2.2.1 UC-01 : Scanner un Fichier . . . 7
2.2.2 UC-02 : Scanner un Dossier . . . 7
2.2.3 UC-03 : Vérifier la Réputation d'une IP ou d'un Domaine . . . 7
2.2.4 UC-04 : Surveiller un Dossier (Analyse Comportementale) . . . 7
2.2.5 UC-05 : Arrêter la Surveillance . . . 7

3 Diagrammes de Séquence 10
3.1 Séquence : Scan d'un Fichier . . . 10
3.2 Séquence : Vérification de Réputation . . . 11
3.3 Séquence : Surveillance Comportementale . . . 12

4 Diagramme de Classes **13**
4.1 Diagramme de Classes Principal 13
4.2 Description des Classes 14
4.2.1 MainWindow 14
4.2.2 ScanWorker (QThread) 14
4.2.3 ReputationWorker (QThread) 14
4.2.4 BehaviorWorker (QThread) 14
4.2.5 ScanResult (dataclass) 14
4.2.6 BehaviorMonitor 14
4.2.7 FileState (dataclass) 14
4.2.8 BehaviorAlert (dataclass) 14

Annexes **15**

Chapitre 1

Cahier des Charges (Version Finale)

1.1 Informations du Projet

Titre du projet	Application de Détection de Malwares
Classe	4 SSIR-E
Étudiant 1	KALAI Hamza
Étudiant 2	OUSSEMA Hamada
Étudiant 3	RAYEN Hassen

Table 1.1 – Équipe du projet

1.2 Introduction Générale et Cadre du Mini Projet

Ce mini-projet fait partie du module **Python appliqué à la cybersécurité**. L'idée est de comprendre, de manière simple, comment on peut analyser des fichiers et reconnaître s'ils contiennent quelque chose de suspect. Le but n'est pas de refaire un vrai antivirus, mais plutôt de voir comment fonctionnent les bases de la détection.

Le but est de développer un outil simple, pédagogique et fonctionnel permettant :

- Une analyse de fichiers par calcul d'empreintes SHA-256
- Une analyse en ligne via l'API VirusTotal (vérification de hash, réputation d'IP et de domaines)
- Une surveillance comportementale (détection d'activités suspectes : création/suppression/modification massive de fichiers)
- Une interface graphique moderne en PyQt5

1.3 Objectifs

1. Réaliser un outil fonctionnel et documenté permettant l'analyse de fichiers et la vérification de réputation d'adresses IP et de domaines.
2. Intégrer des vérifications via l'API publique VirusTotal (analyse de hash SHA-256, réputation IP, réputation de domaines).

3. Fournir une interface graphique moderne en PyQt5 pour l'usage par un utilisateur.
4. Implémenter une surveillance comportementale pour détecter des activités suspectes sur le système de fichiers.

1.4 Besoins Fonctionnels

1.4.1 BF-01 : Scan de fichiers et dossiers

— L'utilisateur peut sélectionner un fichier ou un dossier.
— L'application parcourt récursivement les dossiers et scanne chaque fichier.

1.4.2 BF-02 : Calcul de signatures

— Calcul de l'empreinte SHA-256 pour chaque fichier scanné.
— Comparaison avec la base de données VirusTotal via API.

1.4.3 BF-03 : Analyse de réputation IP/Domaine

— Recherche de la réputation d'une adresse IP via l'API VirusTotal.
— Recherche de la réputation d'un nom de domaine via l'API VirusTotal.
— Affichage clair du résultat (clean/malicious).

1.4.4 BF-04 : Intégration API VirusTotal

— Envoi des hash SHA-256 à l'API VirusTotal pour vérification.
— Gestion des limites de requêtes API (rate limiting).
— Gestion des erreurs réseau et codes de réponse HTTP.

1.4.5 BF-05 : Surveillance comportementale

— Surveillance en temps réel d'un dossier sélectionné.
— Détection des créations, modifications et suppressions de fichiers.
— Alertes automatiques si activité suspecte (type ransomware) détectée.
— Seuils configurables pour les alertes warning/critical.

1.4.6 BF-06 : Journalisation

— Enregistrement des résultats d'analyse dans un fichier de logs.
— Horodatage de chaque événement.

1.4.7 BF-07 : Interface graphique

— Interface PyQt5 moderne avec navigation par onglets.
— Vue scan : sélection fichier/dossier et lancement de l'analyse.
— Vue réputation : vérification d'adresses IP et domaines.
— Vue surveillance : monitoring comportemental en temps réel.
— Barre de progression et affichage coloré des résultats.

1.5 Besoins Non Fonctionnels

1.5.1 Sécurité

— Ne jamais exécuter automatiquement les fichiers scannés.
— Clé API stockée dans un fichier de configuration séparé (`config.py`).

1.5.2 Performance

— Utilisation du traitement parallèle (`ThreadPoolExecutor`) pour les appels API sur plusieurs fichiers.
— Interface graphique réactive grâce à l'exécution des scans dans des threads séparés (`QThread`).

1.5.3 Fiabilité

— Gestion des erreurs réseau (connexion, timeout, codes HTTP 401/404/429).
— Gestion des limites d'API (rate limiting HTTP 429).
— Journalisation des résultats d'analyse dans un fichier de logs (`logs.txt`).

1.5.4 Simplicité

— Interface graphique claire et moderne en PyQt5.
— Navigation intuitive par onglets.
— Affichage en temps réel de la progression des scans.
— Résultats colorés selon le statut (clean/malicious/error).

1.6 Environnements de Développement

1.6.1 Environnement Matériel

Composant	**Spécification**
Machine	PC personnel ou machine de laboratoire
Processeur	Intel i3 ou équivalent
RAM	4 Go minimum
Espace disque	200 Mo minimum

Table 1.2 – Configuration matérielle requise

1.6.2 Environnement Logiciel

1.6.3 Bibliothèques Python Utilisées

Élément	**Spécification**
Plateforme OS	Windows 10/11
Langage	Python 3.10 ou supérieur
Interface graphique	PyQt5
Outils	VS Code / Terminal

Table 1.3 – Environnement logiciel

Bibliothèque	**Usage**
`hashlib`	Calcul d'empreintes SHA-256
`os`	Opérations fichiers et parcours de dossiers
`json`	Traitement des réponses API
`urllib`	Requêtes HTTP vers VirusTotal
`concurrent.futures`	Traitement parallèle (ThreadPoolExecutor)
`dataclasses`	Structure des résultats de scan
`time`, `collections`	Surveillance comportementale
`PyQt5`	Interface graphique

Table 1.4 – Bibliothèques Python utilisées

Chapitre 2

Cas d'Utilisation (Use Cases)

2.1 Diagramme des Cas d'Utilisation

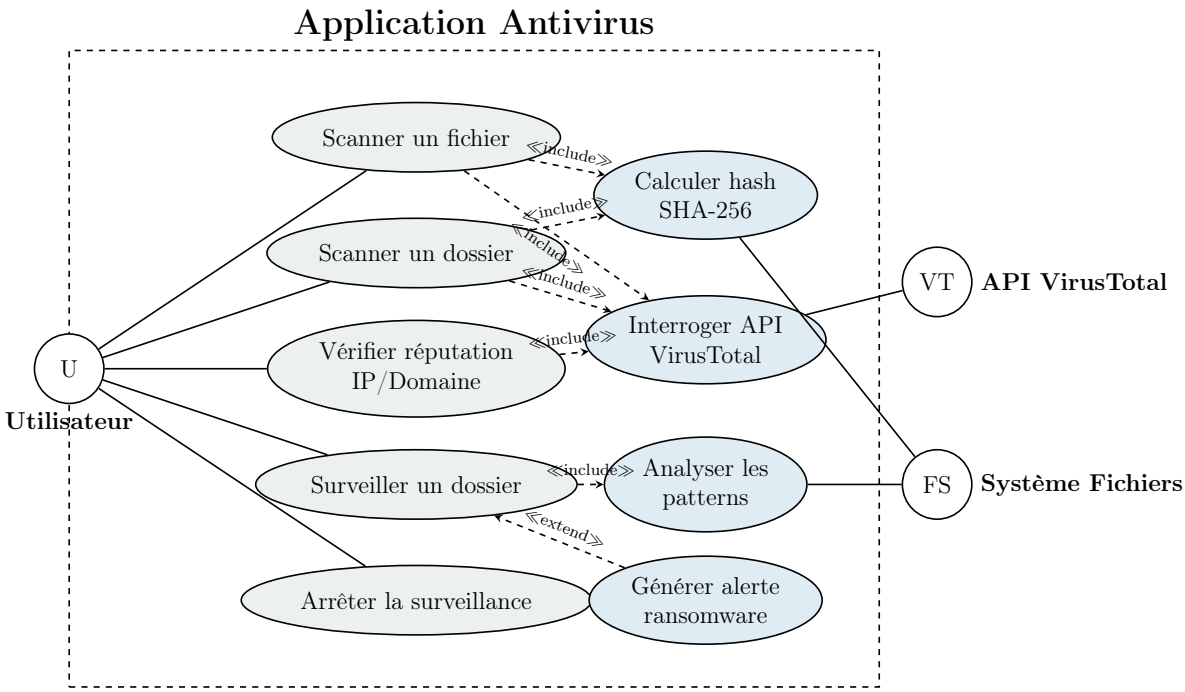

FIGURE 2.1 – Diagramme des cas d'utilisation avec relations ≪include≫ et ≪extend≫

2.2 Description des Cas d'Utilisation

2.2.1 UC-01 : Scanner un Fichier

2.2.2 UC-02 : Scanner un Dossier

2.2.3 UC-03 : Vérifier la Réputation d'une IP ou d'un Domaine

2.2.4 UC-04 : Surveiller un Dossier (Analyse Comportementale)

2.2.5 UC-05 : Arrêter la Surveillance

Élément	**Description**
Acteur principal	Utilisateur
Préconditions	Application lancée, clé API configurée
Scénario nominal	1. L'utilisateur clique sur "Sélectionner Fichier" 2. Il choisit un fichier dans l'explorateur 3. Il clique sur "Scanner Fichier" 4. Le système calcule le hash SHA-256 5. Le système interroge VirusTotal 6. Le résultat s'affiche avec le statut
Postconditions	Résultat affiché, log enregistré
Exceptions	Fichier inaccessible, erreur API, limite atteinte

Élément	**Description**
Acteur principal	Utilisateur
Préconditions	Application lancée, clé API configurée
Scénario nominal	1. L'utilisateur clique sur "Sélectionner Dossier" 2. Il choisit un répertoire 3. Il clique sur "Scanner Dossier" 4. Le système parcourt récursivement les fichiers 5. Pour chaque fichier : calcul hash + interrogation API 6. La progression s'affiche en temps réel 7. Rapport final avec tous les résultats
Postconditions	Tous les fichiers analysés, statistiques affichées
Exceptions	Dossier vide, permissions insuffisantes

Élément	**Description**
Acteur principal	Utilisateur
Préconditions	Application lancée, clé API configurée
Scénario nominal	1. L'utilisateur navigue vers l'onglet "Réputation" 2. Il saisit une adresse IP ou un domaine 3. Il clique sur "Vérifier" 4. Le système détecte le type (IP ou domaine) 5. Interrogation de l'API appropriée 6. Affichage du résultat (clean/malicious)
Postconditions	Réputation affichée
Exceptions	Format invalide, ressource inconnue

Élément	**Description**
Acteur principal	Utilisateur
Préconditions	Application lancée
Scénario nominal	1. L'utilisateur navigue vers l'onglet "Comportement" 2. Il sélectionne un dossier à surveiller 3. Il clique sur "Démarrer" 4. Le système prend un snapshot initial 5. Polling périodique (toutes les secondes) 6. Détection des événements (création/modif/suppression) 7. Alerte si seuils dépassés
Postconditions	Surveillance active, événements loggés
Exceptions	Dossier supprimé pendant la surveillance

Élément	**Description**
Acteur principal	Utilisateur
Préconditions	Surveillance en cours
Scénario nominal	1. L'utilisateur clique sur "Arrêter" 2. Le thread de surveillance s'arrête proprement 3. Les statistiques finales s'affichent
Postconditions	Surveillance arrêtée

Chapitre 3

Diagrammes de Séquence

3.1 Séquence : Scan d'un Fichier

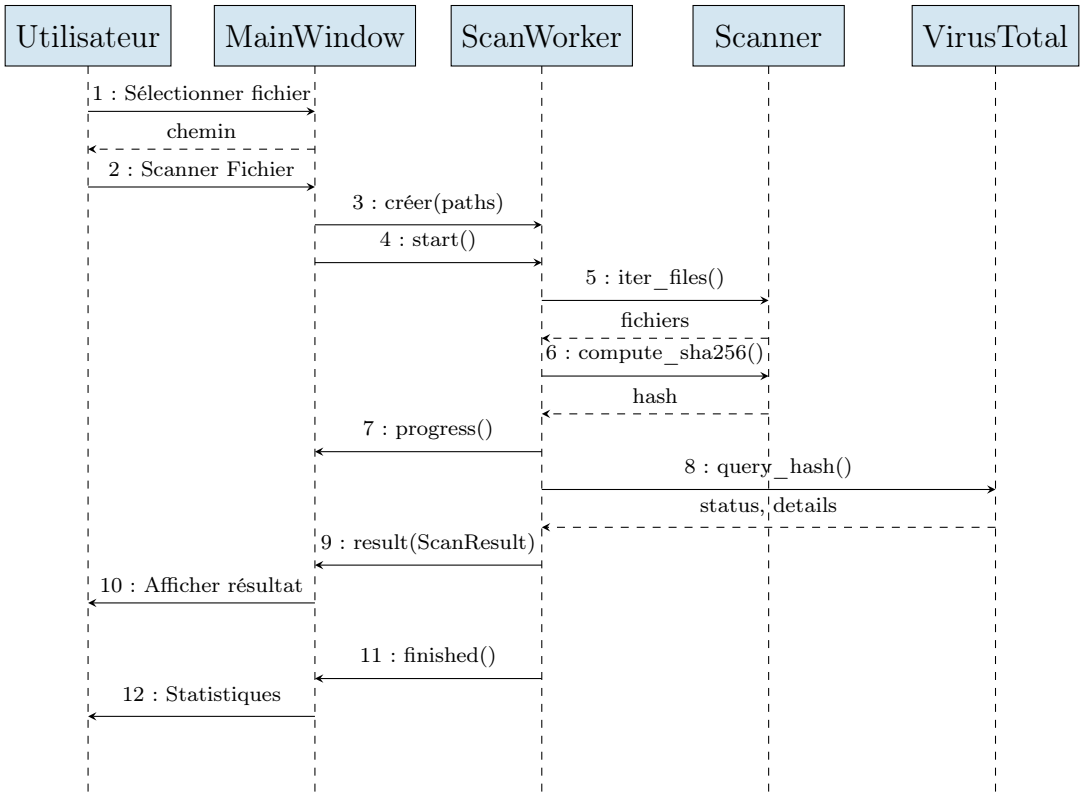

FIGURE 3.1 – Diagramme de séquence - Scan d'un fichier

3.2 Séquence : Vérification de Réputation

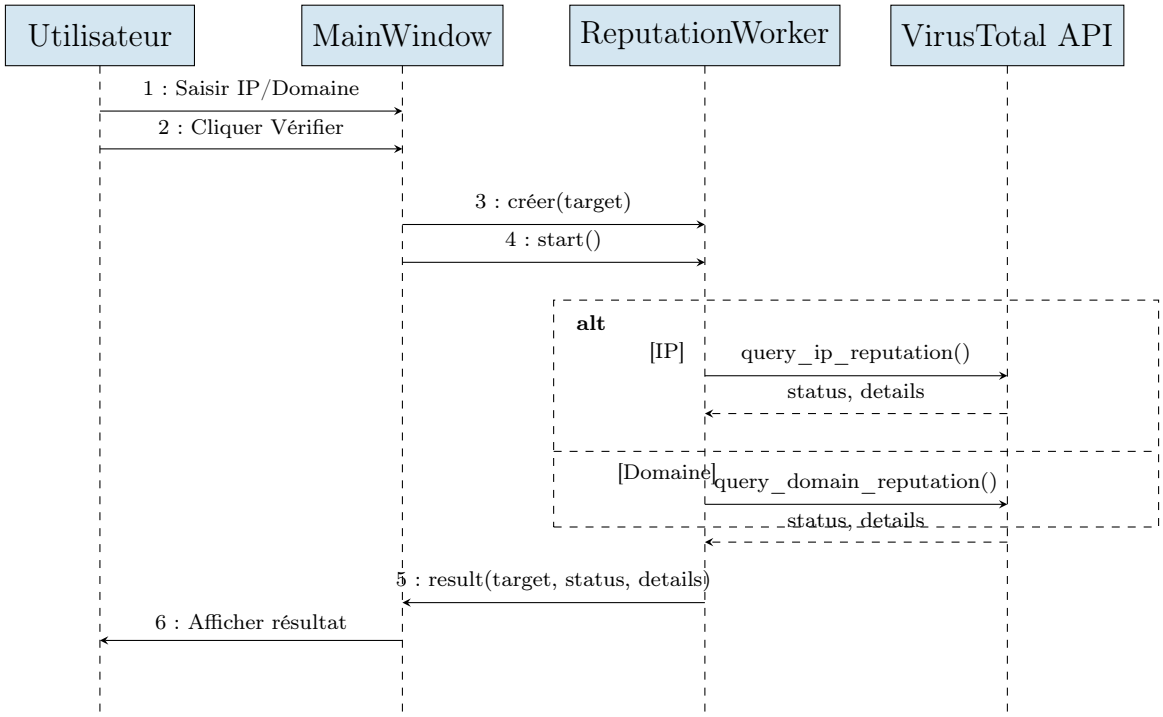

FIGURE 3.2 – Diagramme de séquence - Vérification de réputation

3.3 Séquence : Surveillance Comportementale

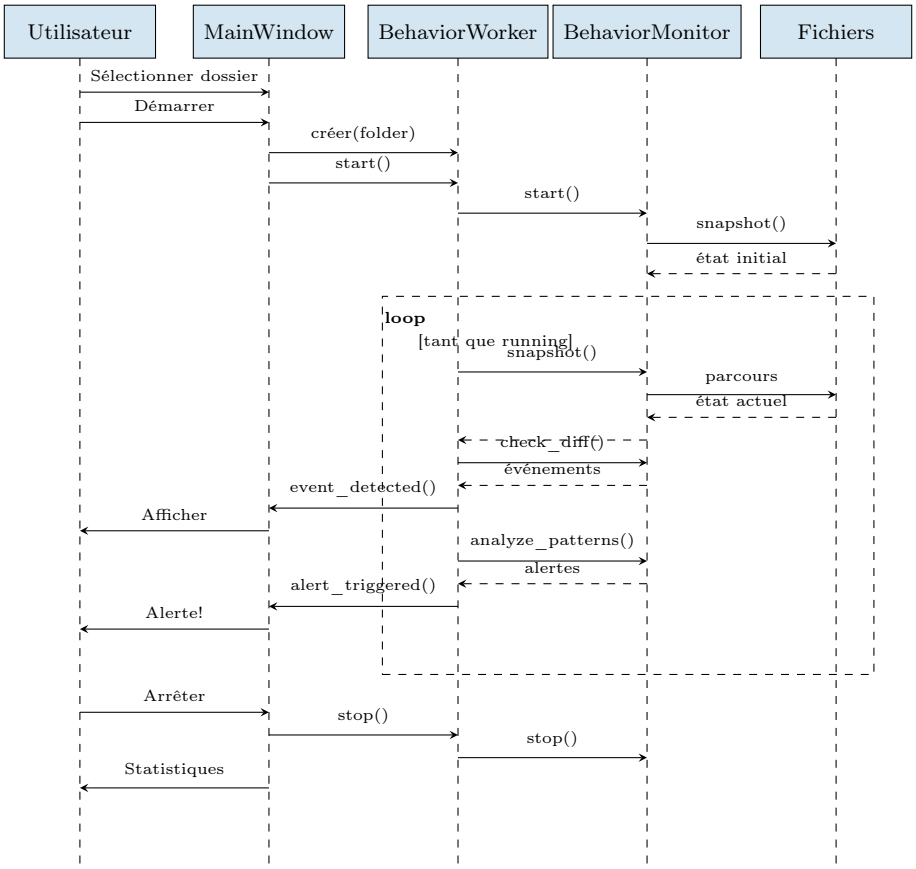

FIGURE 3.3 – Diagramme de séquence - Surveillance comportementale

Chapitre 4

Diagramme de Classes

4.1 Diagramme de Classes Principal

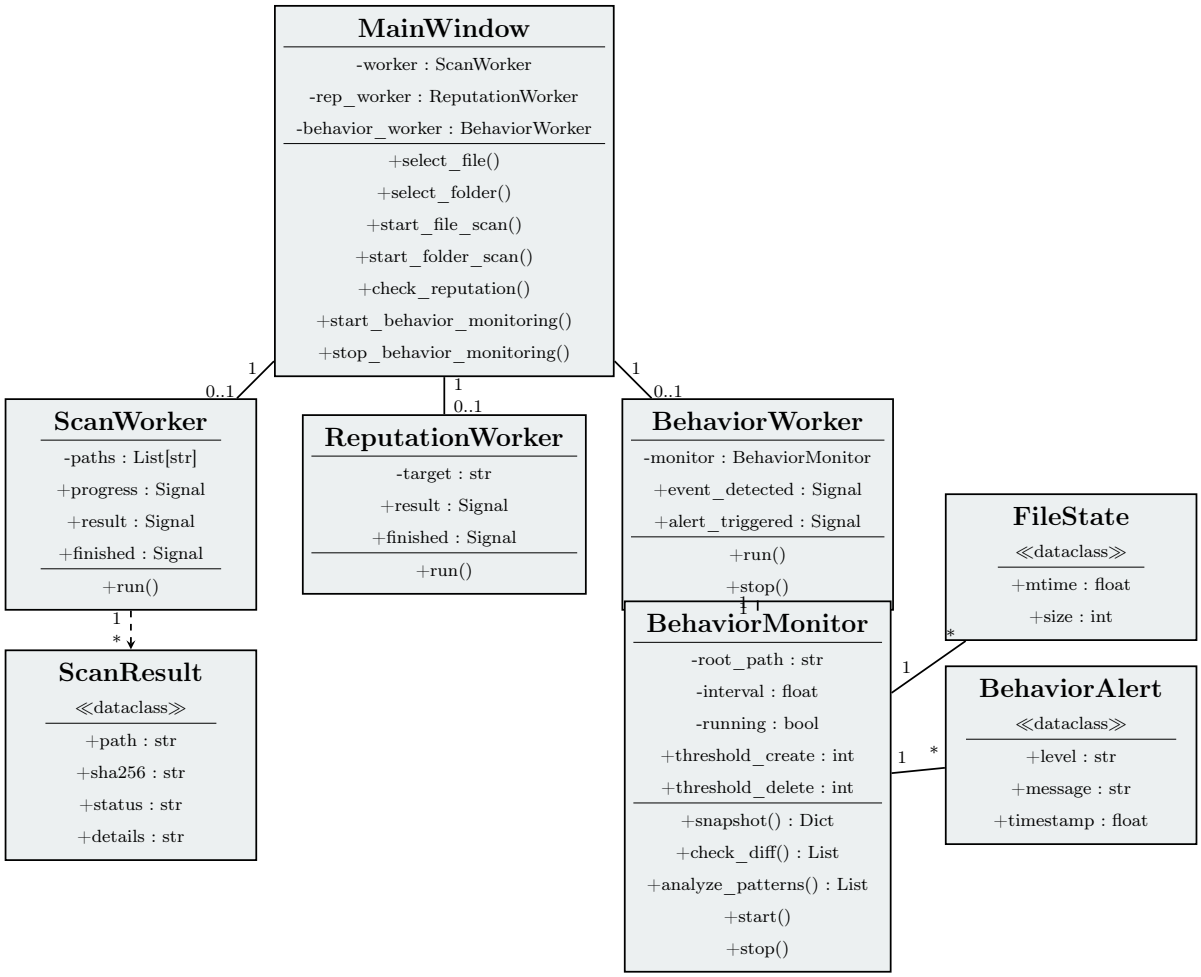

FIGURE 4.1 – Diagramme de classes principal

4.2 Description des Classes

4.2.1 MainWindow

Fenêtre principale de l'application. Gère l'interface utilisateur et coordonne les différents workers pour les opérations asynchrones.

4.2.2 ScanWorker (QThread)

Thread dédié au scan de fichiers. Émet des signaux pour la progression, les résultats et les erreurs.

4.2.3 ReputationWorker (QThread)

Thread dédié à la vérification de réputation IP/Domaine. Détecte automatiquement le type de cible.

4.2.4 BehaviorWorker (QThread)

Thread de surveillance comportementale. Effectue un polling périodique et émet des alertes en cas d'activité suspecte.

4.2.5 ScanResult (dataclass)

Structure de données représentant le résultat d'analyse d'un fichier.

4.2.6 BehaviorMonitor

Classe de surveillance de dossier. Détecte les créations, modifications et suppressions de fichiers et analyse les patterns pour identifier les comportements de type ransomware.

4.2.7 FileState (dataclass)

Snapshot de l'état d'un fichier (date de modification, taille).

4.2.8 BehaviorAlert (dataclass)

Alerte générée par l'analyseur comportemental (niveau warning ou critical).

Annexes

Règles de Détection Comportementale

Événement	**Seuil**	**Fenêtre**	**Niveau**
Créations massives	$\geq$ 5 fichiers	10 secondes	Warning
Suppressions massives	$\geq$ 5 fichiers	10 secondes	**Critical**
Modifications massives	$\geq$ 10 fichiers	10 secondes	Warning

TABLE 4.1 – Seuils de détection comportementale

Statuts de Scan

Statut	**Signification**	**Couleur**
`clean`	Aucune détection malveillante	Vert
`malicious`	Détection(s) malveillante(s)	Rouge
`unknown`	Hash inconnu de VirusTotal	Orange
`error`	Erreur technique	Gris

TABLE 4.2 – Statuts de scan et codes couleur

Document généré le 6 janvier 2026